Short Questions 6

Josh Manchester
Department of Computer Science
University of Colorado Colorado Springs
Colorado Springs, CO 80918
jmanches@uccs.edu

Question 1

What are some similarities and differences between Monte Carlo RL and Temporal Difference RL?

Both Monte Carlo (MC) and Temporal Difference (TD) are model-free methods that learn value functions directly from experience. They share the same incremental update structure: take the current estimate, compare it to some target, and nudge the estimate toward that target using a step size α. So at a high level, both are doing the same thing—correcting predictions based on new information.

The difference is in what we use as that target. MC waits until the end of an episode and uses the actual return G_t (Sutton and Barto 2018). TD does not wait. Instead, it bootstraps after every single step, using $R_{t+1} + \gamma V(S_{t+1})$ as an estimate of the return (Sutton & Barto, Eq. 6.2). This means TD works on continuing tasks and long episodes where waiting for termination is not practical. However, bootstrapping introduces bias because we are substituting part of the return with our own (possibly wrong) estimate $V(S')$. MC avoids this—its estimates are unbiased since they use the true return—but the tradeoff is higher variance, since G_t depends on many stochastic transitions rather than just one.

Question 2

What is the update formula for the famous table-based algorithm called Q-learning? What is the "philosophy" behind how the agent acts and learns in Q-learning?

The Q-learning update (Sutton & Barto, Eq. 6.8) is:

$$Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big] \tag{1}$$

The agent takes action A in state S, gets reward R, and lands in state S'. Then it looks at every action available in S' and picks the one with the highest Q-value—an optimistic prediction of the best thing it could do next. The bracketed term is the TD error: the gap between what we predicted and what we actually observed plus that optimistic next-step estimate.

Q-learning is off-policy. The agent *acts* using an exploratory ε-greedy behavior policy, taking random actions some fraction of the time to discover new information. But it *learns* the optimal policy by always updating toward the greedy action via $\max_a Q(S',a)$, regardless of what action it actually took. This decoupling is what makes it off-policy—we explore freely while our value estimates always reflect the best possible behavior. Unlike off-policy Monte Carlo methods, Q-learning does not need importance sampling because the $\max$ operator directly applies the greedy target policy (Sutton and Barto 2018).

Question 3

To overcome the main shortcoming of Q-learning, researchers have proposed many different approaches. One approach uses two estimators of q values, instead of one. What is the purpose of using two tables instead of one in this modified form of Q-learning? Write down the formulas for updating table entries.

Q-learning has a maximization bias problem. Because the update always takes the $\max$ over estimated action values, it systematically overestimates the true value. Sutton & Barto (p. 134) show a clear example: consider a state where all true action values $q(s,a) = 0$, but our estimates are noisy—some sit above zero, some below. The $\max$ operator picks the highest positive noise and treats it as the value. So we get a positive bias even though the true maximum is zero.

Double Q-learning fixes this by keeping two independent Q-tables, Q_1 and Q_2, and splitting the job in two. One table selects the best action (argmax), and the other table evaluates that action's value. Since selection and evaluation use independent estimates, the positive bias from noisy maximization gets reduced.

The update rules (Sutton & Barto, Eq. 6.10) are applied with equal probability on each step:

With probability 0.5:

$$Q_1(S,A) \leftarrow Q_1(S,A) + \alpha\big[R + \gamma\, Q_2(S', A^*) - Q_1(S,A)\big] \tag{2}$$

where $A^* = \arg\max_a Q_1(S',a)$. Otherwise:

$$Q_2(S,A) \leftarrow Q_2(S,A) + \alpha\big[R + \gamma\, Q_1(S', A^*) - Q_2(S,A)\big] \tag{3}$$

where $A^* = \arg\max_a Q_2(S',a)$.

During episodes, the agent selects actions using ε-greedy over $Q_1 + Q_2$. Both tables individually converge to q_* (Sutton and Barto 2018).

AI Use Statement

Claude (Anthropic) was used to format my answers in AAAI LaTeX format.

References

Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 2nd edition. Available free at http://incompleteideas.net/book/the-book-2nd.html.